

Lesson Notes

MODULE 7 | LESSON 4

HIERARCHICAL RISK PARITY

Reading Time	60 minutes
Prior Knowledge	Traditional portfolio theory and utility theory.
Keywords	behavioral finance, bounded rationality, satisfice, prospect theory, cognitive biases, emotional biases.

Having discussed distance metrics in the context of information and correlation as well as a few important clustering approaches, we combine this knowledge with portfolio construction, especially the risk parity approach to portfolio construction.

1. Risk Parity (RP) and Hierarchical Risk Parity (HRP)

As you might guess from the name, HRP is an extension of risk parity (RP). Recall that RP seeks “to equalize risks or return volatilities of portfolio components” (Sato 6) and avoid some of the pitfalls associated with mean-variance optimization (MVO). As has been discussed in previous lessons, mean-variance optimization (MVO) introduces a large amount of model risk by its reliance on the estimation of returns. With the focus on volatility to the exclusion of returns, RP strategies sidestep this source of error. As a corollary to RP’s goal of equalizing volatility contributions from component assets, allocation increases as the risk of an asset decreases (that is, riskier assets have lower portfolio weights than less risky assets by design). RP’s straightforward diversification technique is therefore at the heart of its risk management approach, which is also the foundation of portfolio construction.

HRP is founded on this RP diversification strategy. Also, like earlier versions of RP, HRP does not incorporate estimated returns and therefore avoids the error contributed by these estimates. Perhaps its main contribution to the RP legacy is its avoidance of the need to invert the covariance matrix of assets—and therefore all the numerical instability that this operation produces. So how does HRP accomplish this? It basically consists of three steps:

1. Clustering
2. Quasi-diagonalization
3. Recursive bisection

2. Step 1: Hierarchical Clustering

Now that we have reviewed RP, let’s discuss the HRP version. The “hierarchical” refers to the type of clustering used to organize the assets. There are three other main types of clustering: partitional, density-based and graph-based (Duarte 110777). You are of course already quite familiar with the clustering of assets, such as stocks, from the last lesson, especially the required reading “A Review of Two Decades of Correlations, Hierarchies, Networks and Clustering in Financial Markets” (Marti et al.) and the direct application of hierarchical clustering in the required reading “Development of Stock Correlation Networks using Mutual Information and Financial Big Data” (Guo et al.). So you already understand that hierarchical clustering “takes into account the fact that complex systems, such as financial markets, can exhibit clear hierarchical structures, which strongly affect the dynamics of complex systems” and “provides a quantitative description of these hierarchies, which can contribute to a more adequate modeling of a system” (Nanakorn 26). As just one example, consider that “financial institutions which have, in the correlation networks, greater node strength, larger node betweenness centrality, larger node closeness centrality and larger node clustering coefficient tend to be associated with larger systemic risk contributions” (Marti 13). Clearly, in this case, clustering captured the centrality of the financial sector to the rest of the financial system. More concretely, hierarchical clustering consists of two main steps:

- (1) Define and estimate the objects’ distance from each other.
- (2) Combine objects into clusters, thus introducing hierarchy.

Now, given your familiarity with RP and clustering, the only thing new about HRP is the combination of the two concepts. Alas, this is not quite the case either because we have actually already worked with a portfolio construction model that uses clustering in order to achieve a version of RP. Recall the required reading from Lesson 3 of Module 5, “A Framework to Perform Asset Allocation Based on Partitional Clustering” (Duarte and De Castro). This was in fact a near relative of HRP, in which the authors substituted *partitional* clustering for *hierarchical* clustering. That was the only substantive difference.

Within hierarchical clustering, there is agglomerative (“bottom-up”) and divisive (“top-down”). Pay attention to the advantages and disadvantages of each of these. Since HRP relies on agglomerative clustering, also pay attention to the procedures and algorithms that are used to determine which clusters should be combined.

The Optimal Number of Clusters algorithm (ONC) helps in this regard by finding the number of clusters such that “each cluster will have high intracluster correlations and low inter-cluster correlations” (de Prado and Lewis 9). It can be summarized as follows:

1. Using historical asset returns, estimate the correlation matrix.
2. Using the correlation matrix, compute the correlation distance matrix.
3. Using the correlation distance matrix, compute the final Euclidean distance matrix.

(Nanakorn 36; de Prado and Lewis 9).

This last step “incorporates information about the entire system, thereby reducing noise and adding robustness to the procedure” (de Prado and Lewis 10).

Also, note that in the last lesson, we mostly focused on the Minimum Spanning Tree (MST), which is a single-linkage tree. That is, we considered the distance between any two clusters to be “the minimum distance between any two objects in the clusters” (Nanakorn 27). Just as there are several distance measures to choose from when building a MST, you will see that there are many ways to define linkages for the sake of constructing a hierarchical cluster, each with their own advantages and disadvantages—to which you should pay attention.

Lastly, on the topic of clustering, de Prado and Lewis enhance (and also complicate) the usually straightforward k-means algorithm due to the two weaknesses they identify: “First, the algorithm requires a user-set number of clusters K , which is not necessarily optimal a priori. Second, the initialization is random, and hence the effectiveness of the algorithm is similarly random” (10). To rectify these problems with k-means, they modify the algorithm by adding a number of iterative and sampling procedures. As you will see in Moen’s repo, there are two main stand-alone functions (and one other function) which help find the optimal number of clusters as suggested by de Prado and Lewis:

- `clusterKMeansBase` performs a “first-pass estimate” of the expected silhouette score for the various numbers of clusters. The k-means algorithm is applied to the Euclidian distance matrix for values of k in the range two to (by default) ten. Since k-means clustering involves random initialization, this procedure is repeated `n_init` times (default is ten) for each k value. The k value with the highest t-value of the Silhouette scores. You will discover the details of calculating the Silhouette score, which measures the difference between average inter-cluster distance and average intra-cluster distance, where a higher Silhouette score suggests a better tree diagram because the clusters are more distinct. (de Prado & Lewis 10-11)
- `clusterKMeansTop` performs a “second-pass estimate” of the expected silhouette score by assessing the *quality* of a given clustering, i.e., the expected Silhouette score divided by the variance of that score over the `n_init` iterations. For those clusters with less than the average quality score, their nodes are re-clustered by applying `clusterKMeansBase` again to this aggregate set of nodes only. If the cluster quality improves, then the output is preserved using `makeNewOutputs`. (de Prado and Lewis 10-11)

Check out this video to get more clarity about how these two functions work within this ONC algorithm.

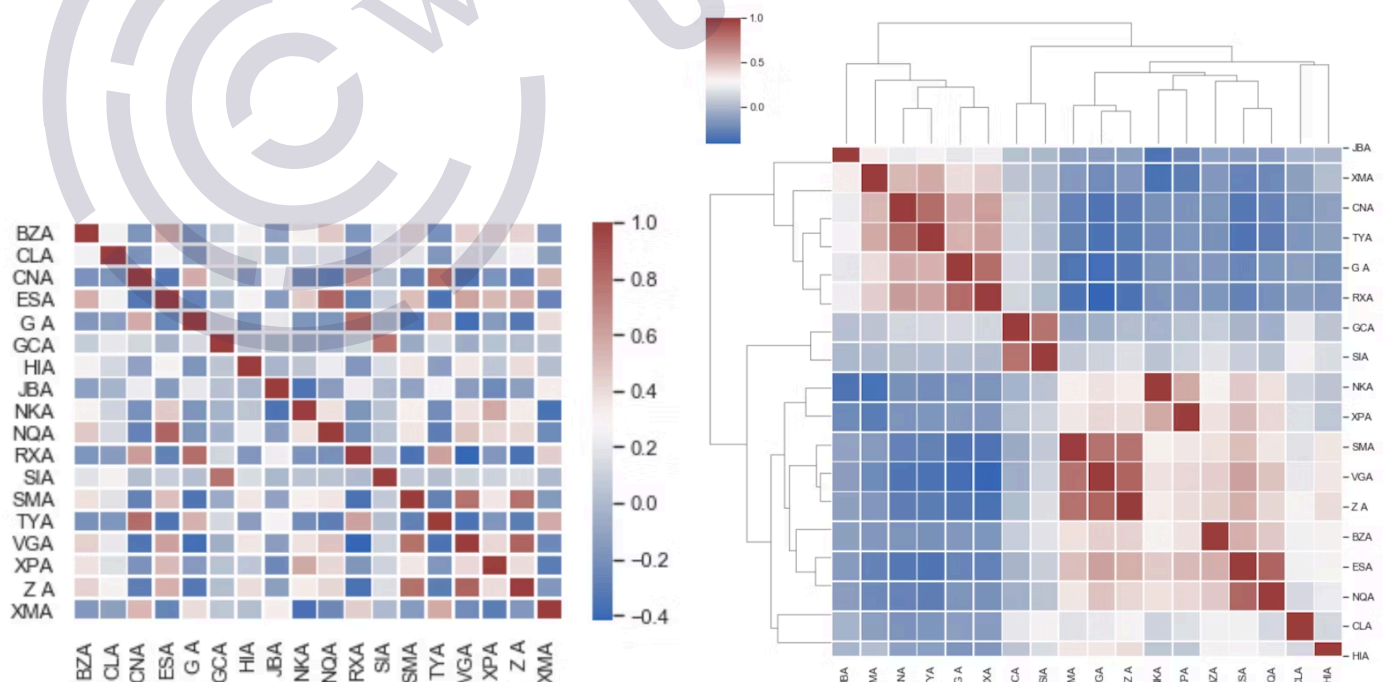
11:41

[Access video transcript here](#)

3. Step 2: Quasi-diagonalization

In a *diagonal* matrix, any non-zero elements are found only along the diagonal and all off-diagonal elements are zero. That is *not* the case here; the *quasi*-diagonalized matrix here simply rearranges the rows and columns of the covariance matrix so that the highest covariances are along the diagonal. There is an algorithm to achieve the proper index to construct the quasi-diagonalized matrix, which you will read about. In short, it requires starting with the last cluster in the cluster tree, replacing it with its constituents, and working up the tree recursively through all the clusters. This resulting sorted list of assets preserves the clustering order and dictates the order of assets for the covariance matrix.

Figure 1: Correlation Matrix, Before and After Clustering and Quasi-diagonalization



Source: Dmitri Marinelli, *Understanding Machine Learning for Diversified Portfolio Construction by eXplainable AI*, <https://oica.isfa.fr/TuesdayAfternoon/Bogota/Marinelli-OICA-slides.pdf>

The following code snippet from 084e0d1c's repo can be used to perform this operation; it can also be used to clarify and confirm your understanding of the pseudo code algorithm presented by Nanakorn (38).

```
def getQuasiDiag(link):

    \# Sort clustered items by distance

    link = link.astype(int)

    sortIx = pd.Series([link[-1, 0], link[-1, 1]])

    numItems = link[-1, 3] # number of original items

    while sortIx.max() >= numItems:

        sortIx.index = range(0, sortIx.shape[0] \times 2, 2) # make space

        df0 = sortIx[sortIx >= numItems] # find clusters

        i = df0.index

        j = df0.values - numItems

        sortIx[i] = link[j, 0] # item 1

        df0 = pd.Series(link[j, 1], index=i + 1)

        sortIx = sortIx.append(df0) # item 2

        sortIx = sortIx.sort_index() # re-sort

        sortIx.index = range(sortIx.shape[0]) # re-index

    return sortIx.tolist()
```

4. Step 3: Recursive Bisection

In this third and final step in the HRP approach to portfolio optimization, we assign weights to asset clusters based on the inverse of their variances. Both Nanakorn and Jain provide pseudo code for the algorithm, so again use the code below to clarify and confirm your understanding.

```
def getRecBipart(cov, sortIx):

    # Compute HRP alloc

    w = pd.Series(1, index=sortIx)

    cItems = [sortIx] # initialize all items in one cluster

    while len(cItems) > 0:
```

```

cItems = [i[j:k] for i in cItems for j, k in ((0, len(i) // 2), (len(i) // 2, len(i)))

for i in range(0, len(cItems), 2): # parse in pairs

cItems0 = cItems[i] # cluster 1

cItems1 = cItems[i + 1] # cluster 2

cVar0 = getClusterVar(cov, cItems0)

cVar1 = getClusterVar(cov, cItems1)

alpha = 1 - cVar0 / (cVar0 + cVar1)

w[cItems0] \times= alpha # weight 1

w[cItems1] \times= 1 - alpha # weight 2

return w

```

5. Innovation and Extension

The research by Jain and by Nanakorn both expand on the original HRP technique. Jain, in particular, employs various methods of specifying the covariance matrix input to HRP and examines the strengths and weaknesses of its performance from the perspective of various metrics (portfolio variance, expected shortfall, Herfindahl index, diversification and Sharpe ratios, etc.) in relation to other portfolio construction approaches (e.g., minimum variance, equal risk contribution, inverse volatility-weighted, and maximum diversification). Nanakorn offers two modified versions of HRP in an effort to address a certain arbitrariness in its methodology, noting that a "reasonable portfolio construction method should allocate the same weight to an asset regardless of where its information is located in the covariance matrix" (40). In addition to the critical and theoretical background to HRP itself that these articles provide, they will hopefully also enhance your appreciation for the proper evaluation of techniques such as HRP as well as innovative approaches to modifying such techniques accordingly.

6. Conclusion

In this lesson, we used what we learned in the previous lesson regarding correlation, information metrics, and clustering to understand and apply HRP and modifications of HRP with the hope of building a better performing portfolio. Indeed, this entire module has focused on improving the primary input to the portfolio optimization process, namely the covariance matrix (de-noising it, de-toning it, making it more robust, etc.). We have also seen a fair amount of testing and comparison of portfolio construction techniques in this lesson as well as the rest of this module. In the next module, we focus more on this testing and evaluation as we explore the hugely important methods of backtesting and avoiding overfitting in the next few lessons.

References

- de Prado, Marcos Lopez and Michael Lewis. "Detection of False Investment Strategies Using Unsupervised Learning Methods." SSRN, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3167017.
- Duarte, Flavio Gabriel, and Leandro Nunes de Castro. "A Framework to Perform Asset Allocation Based on Partitional Clustering." IEEExplore, vol. 8, 2020. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9115637>.
- Jain, Prayut, and Shashi Jain. "Can Machine Learning Based Portfolios Outperform Traditional Risk-Based Portfolios? The Need to Account for Covariance Misspecification." SSRN, 26 May 2019, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3394427.
- Marinelli, Dmitri. Understanding Machine Learning for Diversified Portfolio Construction by eXplainable AI. Lecture. <https://oica.isfa.fr/TuesdayAfternoon/Bogota/Marinelli-OICA-slides.pdf>

- Marti, Gautier et al. "A Review of Two Decades of Correlations, Hierarchies, Networks and Clustering in Financial Markets." Progress in Information Geometry, 2017, pp. 245–274, <https://arxiv.org/abs/1703.00485>.
- Nanakorn, Natasha, and Elin Palmgren. "Hierarchical Clustering in Risk-Based Portfolio Construction." 2021. KTH Royal Institute of Technology School of Engineering Sciences, Independent thesis, Advanced level. <http://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1609991&dswid=3735>
- Sato, Yoshiharu. "Model-Free Reinforcement Learning for Financial Portfolios: A Brief Survey." ArXiv. <https://arxiv.org/pdf/1904.04973.pdf>.

